

# Applying the zero-knowledge proof-of-a-proof to the proof itself

A machine-checked development in Lean 4  
on top of the native Lean 4 Python port

## Abstract

The directory `RequestProject/ZK/` of this project contains a kernel-checked *zero-knowledge proof about a proof*: a prover seals a Python program in a Pedersen commitment and convinces a verifier that a Lean proof of the program’s correctness exists, revealing neither the program nor which catalogue entry it is. This note describes the new layer `RequestProject/ZK/SelfApply.lean`, which applies that construction *to the proof itself*. The hidden object is no longer a program but a *derivation*, checked by an object-level proof kernel formalized here, and the statement the hidden derivation establishes is precisely the soundness claim of the inner zero-knowledge argument. Everything is proved in Lean 4 with no `sorry`, no new axioms, and no `native_decide`: the theorems depend only on `propext`, `Classical.choice` and `Quot.sound`.

## 1 The starting point

The existing layer certifies a statement about a *Python program*. A behavioural specification is a fuel budget, a variable name and a value, and `Meets P S` says that the interpreter of `RequestProject/Python/Core.lean`, started from the empty state, terminates within the budget with that value in that variable. The public data is a Pedersen commitment  $C$  and a catalogue  $\text{cat} : \text{Fin } n \rightarrow \text{Stmt}$  that comes with a Lean proof  $\forall i, \text{Meets } (\text{cat } i) S$ . The certified relation is

$$\text{catRel } g \ h \ \text{cat } C \ (i, r) \iff \text{commitProg } g \ h \ (\text{cat } i) \ r = C,$$

and the OR-composed Schnorr protocol yields an argument system `catArg` for it, with the end-to-end theorem

```
theorem catArg_end_to_end (g h : G) (cat : Fin n → Stmt)
  (S : Spec) (hcat : ∀ i, Meets (cat i) S) (C : G) (p : CatProof n q G)
  (hacc : (catArg g h cat).Accepts C p) :
  ∃ P : Stmt, ∃ r : ZMod q, commitProg g h P r = C ∧
  ∃ σ : State, Python.run S.fuel P = some σ ∧ σ.get S.var = S.value
```

In that picture the *program* is secret and the *proof* is public: the catalogue’s Lean proof `hcat` is quoted in the open. The obvious next move — and the subject of this note — is to hide the proof as well, and to make the hidden proof be a proof *of the statement above*.

## 2 Proofs as objects: a small kernel

To commit to a proof we need proofs to be finite, checkable, Gödel-numberable data. Lean’s own proof terms are not available as first-class data inside Lean, so `SelfApply.lean` formalizes a small object-level calculus and its kernel.

**Definition 2.1** (Syntax). *Formulas are atoms and implications; derivations are hypotheses of an ambient premise list, the two axiom schemes  $K$  and  $S$  of the implicational calculus, and modus ponens.*

```
inductive Form where
  | atom : ℕ → Form
  | imp  : Form → Form → Form

inductive Deriv where
  | hyp : ℕ → Deriv
  | axK : Form → Form → Deriv
  | axS : Form → Form → Form → Deriv
  | mp  : Deriv → Deriv → Deriv
```

**Definition 2.2** (The kernel). *check  $\Gamma$   $d$  returns the conclusion of the derivation  $d$  under the premise list  $\Gamma$ , or `none` if  $d$  is malformed. It is a total, decidable function: an actual proof-checking algorithm.*

```
def check  $\Gamma$  ( : List Form) : Deriv → Option Form
  | .hyp i    =>  $\Gamma$ [i]?
  | .axK a b => some (.imp a (.imp b a))
  | .axS a b c =>
    some (.imp (.imp a (.imp b c)) (.imp (.imp a b) (.imp a c)))
  | .mp d e =>
    match check  $\Gamma$  d, check  $\Gamma$  e with
    | some (.imp p r), some p' => if p = p' then some r else none
    | _, _ => none
```

Formulas are interpreted by a valuation of the atoms, `eval v (.atom i) = v i` and `eval v (.imp a b) = (eval v a → eval v b)`, and the kernel is sound.

**Theorem 2.3** (`check_sound`). *For every valuation  $v$  and premise list  $\Gamma$ , if every premise is true under  $v$  then the conclusion of every checked derivation is true under  $v$ :*

```
theorem check_sound  $\Gamma$  { : List Form} {v : ℕ → Prop}
  (Th : ∀ f ∈  $\Gamma$ , eval v f) :
  ∀ (d : Deriv) (f : Form), check  $\Gamma$  d = some f → eval v f
```

This is what makes the whole construction meaningful: the object-level kernel is not postulated to be sound, its soundness is a Lean theorem, so a checked hidden derivation genuinely transports truth.

### 3 Sealing a proof

The cryptographic layer of the program level is now redone with `Deriv` in place of `Stmt`.

**Definition 3.1** (Gödel numbering and commitment). *codeForm and codeDeriv are injective encodings into  $\mathbb{N}$  built from Cantor pairing; fieldCodeDeriv  $q$  reduces them into  $\mathbb{Z}/q$ , and*

$$\text{commitDeriv } g \ h \ d \ r = \text{code}(d) \cdot g + r \cdot h$$

*is the Pedersen envelope of a proof.*

**Theorem 3.2** (Injectivity, hiding, binding). *codeDeriv\_injective* proves injectivity of the numbering; *commitDeriv\_hiding* proves that the envelope is perfectly hiding (for any two derivations the randomness can be re-mapped bijectively so the envelopes agree); and *commitDeriv\_binds\_proof* proves that, under the discrete-logarithm hypothesis and with both Gödel numbers below the characteristic, a commitment has at most one opening — and that opening determines the derivation itself, not merely its code.

The public statements of the OR-protocol are, for a catalogue  $\text{dcat} : \text{Fin } m \rightarrow \text{Deriv}$  of derivations,

$$Y_i = C - \text{code}(\text{dcat } i) \cdot g,$$

and knowing  $\log_h Y_i$  is exactly knowing an opening of  $C$  to the  $i$ -th derivation (*branchDeriv\_iff*). The relation *proofRel*  $g \ h \ \text{dcat}$  pairs a commitment with such an opening, and

```
noncomputable def proofArg [NeZero m] [Fact (Nat.Prime q)]
  (g h : G) (dcat : Fin m → Deriv) :
  ArgSystem G (Fin m × ZMod q) (proofRel g h dcat)
```

is the concrete argument system built from the OR-composed  $\Sigma$ -protocol, with completeness (*proofArg\_complete*), knowledge soundness via the rewinding extractor, and the recursive form *proofArg\_end\_to\_end\_recursive* (a proof about a proof about a proof, to any depth).

**Theorem 3.3** (The semantic payoff, *proofArg\_transports\_truth*). *If every catalogue entry is a checked derivation of target and the premises are true under a valuation v, then acceptance of the outer transcript entails both that a kernel-checked derivation of target is sealed in C, and that target is true:*

```
∃( d : Deriv, ∃ r : ZMod q,
  commitDeriv g h d r = C ∧ check Γ d = some target )
  ∧ eval v target
```

Two hiding results delimit what the verifier learns: *proofArg\_hides\_derivation* (any transcript produced from one catalogue derivation could have been produced, with other randomness, from any other) and *sealed\_proof\_hidden* (the envelope alone is perfectly hiding).

## 4 The self-application

Everything so far is generic. The self-application is the choice of premises, target and valuation.

Take two atoms:

- atom 0 = “the program-level verifier accepted the transcript  $p$ ”;
- atom 1 = “the commitment  $C$  opens to a program that meets the specification”.

The premise list is

$$\Gamma = [\text{atom } 0 \rightarrow \text{atom } 1, \text{atom } 0], \quad \text{target} = \text{atom } 1.$$

Read through the intended valuation *selfVal*, the first premise is the Lean theorem *catArg\_end\_to\_end* of Section 1, and the second is the hypothesis that the inner argument was accepted; this is the content of *selfPremises\_valid*. The object-level statement whose proof we now hide is therefore exactly the soundness claim of the inner zero-knowledge argument.

The catalogue holds two genuinely different derivations of that statement:

```

def dDirect : Deriv := .mp (.hyp 0) (.hyp 1)

def dDetour : Deriv :=
  .mp (.hyp 0)
    (.mp (.mp (.axK accepted (.imp accepted correct)) (.hyp 1)) (.hyp 0))

```

The first is one modus ponens; the second takes a detour through the axiom scheme  $K$ , deriving  $A$  again from  $A$  and  $A \rightarrow B$  before concluding. `selfCat_checks` proves by kernel computation that both check to the target, and `selfCat_distinct` that they are different proofs (their Gödel numbers are 13 and a 123-digit number).

**Theorem 4.1** (`zkpop_of_zkpop`). *Let  $\text{cat}$  be a catalogue of Python programs with a Lean proof that each meets the specification  $S$ ; let  $C$  be a commitment and  $p$  a transcript accepted by the program-level verifier; let  $D$  be a commitment and  $pp$  a transcript accepted by the proof-level verifier for the catalogue  $\{\text{dDirect}, \text{dDetour}\}$ . Then*

$$\begin{aligned}
& \exists (d : \text{Deriv}, \exists r : \text{ZMod } q, \text{commitDeriv } g \ h \ d \ r = D \wedge \\
& \quad \text{check selfPremises } d = \text{some selfTarget}) \wedge \\
& \exists P : \text{Stmt}, \exists r : \text{ZMod } q, \text{commitProg } g \ h \ P \ r = C \wedge \\
& \exists \sigma : \text{State}, \text{Python.run } S.\text{fuel } P = \text{some } \sigma \wedge \sigma.\text{get } S.\text{var} = S.\text{value}
\end{aligned}$$

The verifier therefore learns three things and no more:

1. a kernel-checked *proof* of the inner argument’s soundness claim is sealed in  $D$  — but not which of the two proofs it is;
2. that claim holds: the *program* sealed in  $C$  terminates within its fuel budget with the specified value — but not which program it is;
3. nothing else: both envelopes are perfectly hiding, and both transcripts are witness-indistinguishable.

It is worth stressing how the second item is obtained. It is *not* proved by appealing to `catArg_end_to_end` directly; it is obtained by running the object-level kernel’s soundness theorem on the hidden derivation extracted from the outer argument. The truth of the inner claim travels through the sealed proof. This is the precise sense in which the zero-knowledge proof-of-a-proof has been applied to the proof itself.

*Remark 4.2* (Two kinds of recursion). The layer `ArgSystem.lean` already provides *protocol* recursion: `recurse` wraps an argument system inside a verifier for its own proofs, and `tower` iterates that to any depth, preserving completeness and knowledge soundness. The self-application of this note is *semantic* recursion: the content of the outer statement is the correctness of the inner protocol. The two compose — the recursive form `proofArg_end_to_end_recursive` applies verbatim to the self-application’s catalogue.

## 5 A runnable instance

Over the Mersenne prime  $q = 2^{31} - 1$ , with the same generators as the program-level demo, the file seals the derivation `dDirect` with randomness `selfRand` and runs the protocol. The following are theorems, not tests:

---

|                                    |                                                                 |
|------------------------------------|-----------------------------------------------------------------|
| <code>selfCat_checks</code>        | both hidden proofs are kernel-checked derivations of the target |
| <code>selfCat_distinct</code>      | the two catalogue entries are different proofs                  |
| <code>selfY_zero</code>            | the prover really knows the opening it claims                   |
| <code>selfProof_verifies</code>    | the honest outer proof is accepted, for <i>every</i> challenge  |
| <code>selfNIPProof_verifies</code> | so is its non-interactive (Fiat–Shamir) form                    |
| <code>selfForgery_rejected</code>  | a transcript with a tampered challenge is rejected              |

---

`RequestProject/Main.lean` additionally executes the verifier: it prints `true` for the honest interactive and non-interactive proofs, `false` for the tampered transcript, and `false` for a prover claiming the branch it cannot open.

## 6 What is not claimed

- Soundness of the Fiat–Shamir transform (a random-oracle assumption) is not formalized; only its completeness is, and `demoHash` is a toy hash used for illustration.
- Binding of both envelopes is conditional, as it must be, on the discrete logarithm of  $h$  in base  $g$  being unavailable, and on the Gödel numbers concerned being smaller than the field characteristic. The runnable instance uses  $2^{31} - 1$ , which carries the protocol but is far too small to separate the codes of the two catalogue derivations; the theorems it instantiates (completeness, rejection of forgeries, hiding) do not depend on that bound.
- Knowledge soundness is the standard special-soundness packaging: the proof object consumed by the extractor is the pair of accepting transcripts a rewinding extractor obtains.
- The object-level calculus is the implicational fragment with two axiom schemes and modus ponens. It is expressive enough to carry the reasoning step that the self-application needs — and, crucially, its kernel’s soundness is proved rather than assumed — but it is not a formalization of Lean’s own kernel.
- As `acceptance_alone_insufficient` records in `ArgSystem.lean`, the argument layer *transports* mathematical content, it never creates it. All the content here comes from Lean-checked theorems: about the Python interpreter at the inner level, and about the object-level kernel at the outer one.

## Provenance

All statements in this note are Lean 4 declarations in `RequestProject/ZK/SelfApply.lean`, building against `Mathlib`. The module contains no `sorry`, no `admit` and no new axioms, and no `native_decide`: every theorem cited above depends only on `propext`, `Classical.choice` and `Quot.sound`.